

- $TGS_SessionKey = Decrypt [Auth_Response, Client_LongLivedKey]$
- Client cannot decrypt the TGT as it does not have TGS-LongLivedKey with it.
The status of various keys at the end of client authentication with authentication server is as follows.
Client will have the following keys:
 - Client-LongLivedKey
 - TGS-SessionKey
 Authentication server will have the following keys:
 - TGS-SessionKey
 - Client-LongLivedKey
 - TGS-LongLivedKey
 TGS component will have the following keys:
 - TGS-LongLivedKey
 Service will have the following keys:
 - Service-LongLivedKey
- $TGS_Req = Authenticator_client + TGT$
- Client sends the request to TGS.
- TGS processes the TGT and decrypts it using TGS-LongLivedKey to extract the TGS-SessionKey from TGT.
 - $TGS_SessionKey = Decrypt [TGT, TGS_LongLivedKey]$
- TGS decrypts the authenticator sent by client, to confirm the client identity:
 - $Decrypt [Authenticator_client, TGS_SessionKey]$
- If it can decrypt successfully, the client is authentic as the TGT is provided by a trusted KDC component, i.e., the authentication server. TGS then proceeds further to respond to the client.
- TGS generates the service session key (in this case the file server), and encrypts the session key using TGS-SessionKey. Additionally, TGS creates the service ticket to be used by the service (file server) for confirming the validity of the client.
 - $Authenticator_TGS = Encrypt [Service_SessionKey, TGS_SessionKey]$
 - $ServiceTicketInPlain = (ClientId + Client IP + Service_SessionKey + Expiry information of Service Ticket)$
 - $ServiceTicketEncrypt = Encrypt [ServiceTicketInPlain, Service_LongLivedKey]$
- TGS sends the Authenticator-TGS + ServiceTicketInEncrypt.
- Client receives the response from the TGS, and processes it to fetch the Service-SessionKey by decrypting an Authenticator-TGS using TGS-SessionKey:
 - $Service_SessionKey = Decrypt [Authenticator_TGS, TGS_SessionKey]$
- Client cannot decrypt the service ticket as it is encrypted using Service-LongLivedKey. Client does not have this key with it to decrypt the service ticket.

The status of various keys at the end of client authentication with TGS is as follows.

Client will have the following keys:

- Client-LongLivedKey
- TGS-SessionKey
- Service-SessionKey

Authentication server will have the following keys:

- TGS-SessionKey
- Client-LongLivedKey
- TGS-LongLivedKey

TGS component will have the following keys:

- TGS-LongLivedKey
- Service-SessionKey
- TGS-SessionKey
- Service-LongLivedKeys

Service will have the following keys:

- Service-LongLivedKeys

Client service communication

In this stage, the client has the service session key and the service ticket, which it uses to make requests for service. The service session key is used for encrypted communication in this stage. Figure 3 gives a schematic representation of the information flow.

- Client generates an authenticator using time stamp, and encrypts it using Service-SessionKey:
 - $Authenticator = Encrypt [Timestamp, Service_SessionKey]$
- Client requests service (file server) with the following payload and sends it to service:
 - $Request = (Authenticator + ServiceTicketEncrypt received in stage 2)$
- Service decrypts the service ticket using Service-LongLivedKey as:
 - $Service_SessionKey = Decrypt [ServiceTicketEncrypt, Service_LongLivedKey]$
- Service decrypts an authenticator using Service-SessionKey as:
 - $Decrypt [Authenticator, Service_SessionKey]$

The status of various keys at the end of service client communication is as follows.

Client request with KDC for service

In this stage, the TGS verifies the client, based on the authenticator encrypted by the client using TGS-SessionKey. The TGS, after successfully verifying the client, generates the session key to be used when communicating with service (say, file server in the Kerberos infrastructure) for the client. The process is represented schematically in Figure 2.

The steps for stage 2 are given below.

- From stage 1 above, where the client is authenticated by the authentication server, the client now has a TGS session key with it and the TGT. The client cannot decrypt the TGT as it is encrypted by TGS-LongLivedKey. The client does not have TGS-LongLivedKey (refer to the key status section above in stage 1).
- Client creates an authenticator using the time stamp of the machine and encrypts it using TGS-SessionKey.
 - $Authenticator_client = Encrypt [TimeStamp, TGS_SessionKey]$
 - Client forms the payload in two parts — one is an authenticator and the other is TGT (encrypted using TGS-LongLivedKey in stage 1).